Full-Stack Accelerator Evaluation on HDnn-PIM

Chi-Han Chiu
Computer Engineering, University of California, San Diego
c8chiu@ucsd.edu

***Abstract*—The integration of Hyperdimensional Computing and Processing-in-Memory (HDnn-PIM) offers exceptional energy efficiency for edge computing. However, existing Design Space Exploration (DSE) methods rely predominantly on inaccurate software-level analytical models, while incorporating actual RTL synthesis incurs prohibitive time overheads. To bridge this gap, we propose an automated full-stack co-design framework with gate-level fidelity. Our approach unifies software hyperparameters, hardware dimensions, and EDA synthesis flags into a single, comprehensive search space. To overcome evaluation bottlenecks, we introduce a constraint-aware, multi-fidelity evaluation pipeline equipped with early-stage gatekeeping and fast black-box synthesis. By extracting Energy , Performance, and Area (PPA) metrics via logic synthesis and driving power estimation using actual gate-level waveform switching activities, our framework significantly surpasses the accuracy of pure software simulations and static post-synthesis analyses. Experimental results show that the framework rapidly prunes invalid designs and accelerates exploration. Compared to isolated evaluations, it successfully unearths gate-level verified Pareto-optimal trade-offs—minimizing energy, timing, and area overheads, thereby drastically improving the design efficiency of HDnn-PIM architectures.**

***Index Terms*—Hardware Accelerator, Hyperdimensional Computing, Processing-in-Memory, Design Space Exploration, Customizable Pipeline, Gate-Level Simulation, Full-Stack Co-Design.**

I. Introduction

With the proliferation of Artificial Intelligence (AI) and edge computing, domain-specific accelerators must meet increasingly stringent Power, Performance, and Area (PPA) constraints. Modern accelerator design demands a synergistic, cross-layer co-design approach spanning software algorithms, hardware architectures, and Electronic Design Automation (EDA) synthesis strategies. However, acquiring high-fidelity PPA metrics typically requires exhaustive RTL logic synthesis and gate-level simulation. This significant computational overhead creates a major evaluation bottleneck for traditional Design Space Exploration (DSE), especially when high-dimensional parameters are involved.

Two primary challenges impede optimal accelerator design. First, **cross-layer design space coupling** creates nonlinear interdependencies between model, architecture, and synthesis parameters. Traditional decoupled optimization, where software hyperparameters are fixed prior to hardware design, ignores the mutual constraints between layers: hardware efficiency is sensitive to algorithmic configurations, while software performance is bounded by physical resources. For instance, increasing the model parameter `hd_dim` may exceed the memory capacity of a hardware tile, resulting in significant performance degradation. Consequently, reaching the global Pareto frontier necessitates *joint parameter tuning* to balance algorithmic accuracy against hardware constraints. Table I details the specific search variables instantiated in our HDnn-PIM [1] case study.

TABLE I: Representative Cross-Layer Search Variables in the Proposed Flow (Illustrated via the HDnn-PIM Case Study)

Level	Parameters	Impact
Model / Software	D (`hd_dim`), `inner_dim`, `out_channels`	Determines accuracy threshold; nonlinearly coupled with HW resources.
Hardware Architecture	`reram_size`, `encoder_x/y_dim`, `frequency`	Direct physical impact on Area, Power, and Timing boundaries.
Synthesis Strategy	`enable_retime`, `syn_map_effort`, `clock_gating`	Dynamically steers physical optimization via synthesis flags.

Second, there is a **lack of an integrated multi-fidelity evaluation pipeline** . Within a search space exceeding 10^5 combinations, executing 40–90 minute RTL evaluations per candidate is computationally prohibitive. Existing DSE literature either heavily relies on pure analytical models [2], [3] or targets decoupled EDA tuning [4], [5]. A reusable evaluation flow that orchestrates software screening, physical synthesis, and gate-level verification is still lacking in the current landscape.

To overcome these evaluation bottlenecks, we propose the highly configurable, gated multi-fidelity evaluation framework illustrated in Fig. 1. This methodology introduces an automated full-stack co-design pipeline. By deploying a "Software $\rightarrow$ Synthesis $\rightarrow$ Gate-Level" gated mechanism, it leverages software accuracy boundaries and hardware timing violations as early-stopping conditions. This proactively prunes substandard designs, maximizing the computational value of expensive RTL evaluations. We validate this generic methodology on HDnn-PIM, a highly coupled cross-layer architecture.

The primary contributions of this work are summarized as follows:

- **Unified Cross-Layer Parameterization Interface:** We unify algorithmic hyperparameters, architectural dimensions, and physical EDA directives (incorporating over 13 Design Compiler flags) into a single optimization space.
- **Gated Multi-Fidelity Evaluation Pipeline:** We design a strict three-stage hierarchy. Gatekeeping utilizes software accuracy (Gate 1) and timing violations (Gate 2) for early termination, profoundly pruning the search space and bypassing massive synthesis overheads.

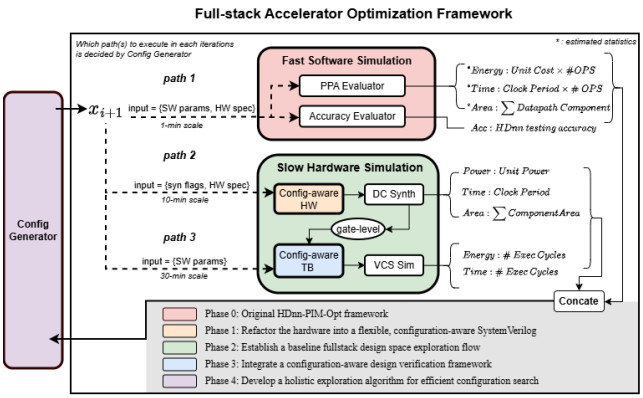


Fig. 1: Overview of the proposed full-stack evaluation flow with a gated multi-fidelity pipeline.

- **Automated EDA-in-the-Loop Orchestration:** We seamlessly integrate heavy RTL synthesis and gate-level simulations into the iterative search loop. Crucially, the final simulation stage leverages actual waveform toggle behaviors for power analysis, yielding energy predictions with high fidelity.

The remainder of this article is organized as follows: Section II introduces the validation case study and related work. Section III details the multi-fidelity framework and EDA orchestration strategy. Section IV analyzes the experimental results. Section V discusses limitations, and Section VI concludes the paper.

II. Background and Related Work

This section establishes the rationale for the selected validation vehicle, delineates the fundamental bottlenecks of physical RTL feedback, and positions our methodology within the broader research landscape.

A. HDnn-PIM

To rigorously validate the proposed methodology, the target accelerator must exhibit severe nonlinear coupling across multiple abstraction tiers. The integration of Hyperdimensional Computing and Processing-in-Memory (HDnn-PIM) [1] serves as an optimal experiment target.

In HDnn-PIM, algorithmic hyperparameters strictly dictate physical memory capacities, which directly define hardware power limits, timing boundaries, and routing congestion. Consequently, isolated layer optimization invariably fails to approximate the global Pareto frontier. Optimizing this tightly coupled system across all tiers simultaneously provides a robust stress-test for our automated orchestration flow.

B. RTL Evaluation Bottlenecks

Exploring high-dimensional design spaces ($\sim 10^5$ points) is bottlenecked by the high cost of hardware feedback. While fast, analytical models often neglect physical implementation details like wire delays and post-routing area. In contrast, RTL-based PPA extraction is computationally expensive; empirical measurements show that even for minimal configurations, synthesis requires approximately 7 minutes, compared to only 30 seconds for software-level evaluation. To address this $10\times$ disparity, we propose a gated, multi-fidelity flow. By utilizing software accuracy and hardware timing violations as early-exit conditions, the framework prunes the search space and maximizes exploration throughput without sacrificing final PPA fidelity.

C. Limitations of Prior Work

Existing Design Space Exploration (DSE) methodologies generally exhibit a fundamental trade-off between evaluation fidelity and optimization scope, as summarized in Table II.

From a top-down perspective, frameworks designed for rapid architectural exploration [2], [3] depend heavily on unified analytical models. By abstracting away logic synthesis, these methodologies inherently fail to capture localized physical realities, such as wire delays and routing congestion. Even when automated search algorithms are introduced, as seen in HierCGRA [7], the exploration is predominantly restricted to architectural parameters, leaving underlying EDA synthesis directives unexplored. Consequently, domain-specific accelerators like the original HDnn-PIM [1] are frequently constrained to static, sub-optimal manual configurations.

Conversely, from a bottom-up perspective, contemporary physical design methodologies [4], [5] have definitively established the critical impact of EDA parameter tuning on final PPA metrics. However, these optimizations operate in strict isolation at the logic or physical tiers, entirely disregarding synergistic trade-offs with algorithmic hyperparameters. Recent attempts to bridge this cross-layer divide, such as the multi-objective co-optimization framework for HD-PIM [6], remain analytically constrained. Because their evaluation relies on software-level simulations rather than automated RTL synthesis and waveform-driven gate-level verification, they ultimately fail to close the abstraction gap, leaving the true physical Pareto frontier out of reach.

D. Methodological Positioning

Rather than proposing an isolated accelerator optimization or a standalone EDA tuning algorithm, this work introduces a generalized, multi-fidelity orchestration framework that bridges the abstraction gap between predictive modeling and physical realization.

We advance beyond existing co-design methodologies [6] through three key capabilities: (1) integrating fine-grained EDA synthesis directives into a unified cross-layer search space; (2) extracting high-fidelity PPA metrics via automated RTL synthesis and waveform-driven gate-level simulation; and (3) providing a modular, architecture-agnostic pipeline. We substantiate the robustness of this framework by instantiating it on the tightly coupled HDnn-PIM architecture.

III. Proposed Full-Stack DSE Framework

To bridge the abstraction gap between analytical modeling and physical synthesis, we propose a multi-fidelity orchestration framework. This system integrates algorithmic hyperparameters, hardware configurations, and EDA synthesis directives into a unified, cross-layer search space. The framework is

TABLE II: Positioning of the Proposed Methodology Against State-of-the-Art Literature

Related Work	SW/HW Co-Design	EDA Strategy Exploration	Gate-Level Fidelity	RTL Eval Acceleration	Customizable Pipeline	Full-Stack
Timeloop [2]	×	×	Analytical Models	✓	×	×
CiMLoop [3]	×	×	Analytical Models	✓	×	×
HDnn-PIM [1]	Partial	×	—	×	×	×
Yang et al. [6]	✓	×	Simulation Only	Partial	×	✓
HierCGRA [7]	✓	×	Logic Synthesis	Partial	Partial	×
Sun et al. [4]	×	✓	Logic Synthesis	Partial	×	×
REMOTune [5]	×	✓	Logic Synthesis	Partial	×	×
Proposed Work	✓	✓	**Gate-Level Sim.**	✓**(Gated Mode)**	✓	✓

Note: "Partial" indicates limited coverage or lack of early-stopping. "Simulation Only" indicates the absence of actual physical synthesis.

built upon a modular philosophy, allowing for flexible adaptation of optimizers, evaluation stages, and physical constraints across diverse accelerator architectures.

A. Decoupled Thin-Client Architecture

A significant barrier to integrating modern Python-based optimization engines (e.g., Ax, BoTorch) with commercial EDA tools (e.g., Synopsys Design Compiler, VCS) is the restriction of enterprise licensing and intranet firewalls. We circumvent these bottlenecks through a decoupled thin-client architecture, as illustrated in Fig. 2.

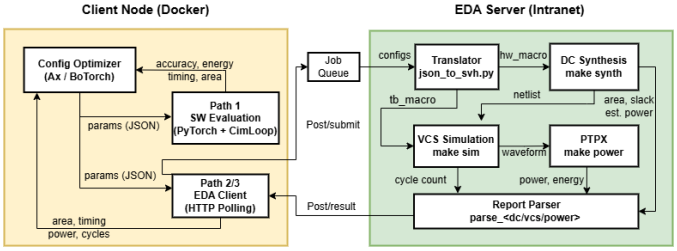


Fig. 2: Remote deployment architecture of the full-stack DSE framework with a thin-client interface.

The client-side engine encapsulates candidate parameters into JSON payloads, which are transmitted to a remote EDA server via an asynchronous polling protocol. The server automates the RTL generation, synthesis orchestration, and log parsing (extracting Area, Slack, and Power via regex), returning only the processed PPA metrics. This architecture ensures scalability and prevents connection timeouts during long-running synthesis tasks.

B. Gated Multi-Fidelity Pipeline

Evaluating a single candidate via full RTL simulation can exceed 90 minutes. To optimize computational resources, we implement a three-stage evaluation hierarchy with early-stopping gates.

1) Path 1: Algorithmic Screening: The framework first evaluates the model's accuracy via software simulation (e.g., PyTorch) and estimates analog RRAM metrics. This stage enforces *Gate 1*: if the candidate fails to meet a predefined accuracy threshold τ, the evaluation is immediately terminated, bypassing all subsequent physical design stages.

2) Path 2: Logic Synthesis and Hybrid Stitching: Candidates passing Gate 1 proceed to RTL synthesis. To manage the lack of standardized RTL for analog RRAM arrays, we utilize a hybrid stitching strategy to estimate the total area (A_{total}), timing (T_{total}), and energy (E_{total}):

$$\begin{aligned} A_{\text{total}} &= A_{\text{ASIC}} + A_{\text{RRAM}} \\ T_{\text{total}} &= T_{\text{ASIC}} + T_{\text{RRAM}} \\ E_{\text{total}} &= P_{\text{ASIC}} \cdot T_{\text{total}} + E_{\text{RRAM}} \end{aligned} \tag{1}$$

Gate 2 acts as a physical feasibility checkpoint. If the synthesis results in a negative timing slack (Slack < 0), the design is discarded, and Path 3 is not triggered.

3) Path 3: Waveform-Driven Verification: Physically viable candidates undergo VCS gate-level simulation and PrimeTime PX (PtPX) analysis. To minimize network overhead, we employ a local LFSR-based self-generating testbench on the server. This generates pseudo-random stimuli that statistically mirror inference workloads, providing high-fidelity dynamic and leakage power metrics based on actual circuit switching activities:

$$\begin{aligned} T_{\text{ASIC}} &= (T_{\text{clk}} \times N_{\text{cycles}})/1000 \quad (\mu\text{s}) \\ E_{\text{ASIC}} &= (P_{\text{dyn}} + P_{\text{leak}}) \times T_{\text{ASIC}} \end{aligned} \tag{2}$$

For rapid exploration, a *fast black-box mode* is provided, allowing the compiler to abstract parameter-independent modules and focus exclusively on core logic.

C. Cross-Layer Design Space Definition

The framework dissolves the traditional boundaries between architectural DSE and physical synthesis. High-level software and hardware parameters are systematically mapped to RTL macros, while fine-grained EDA flags are integrated directly into the Bayesian Optimization search space.

Table III summarizes the primary search space dimensions. The underlying technical mapping logic and the complete set of 13+ synthesis directives (e.g., clock gating, retiming, and effort levels) are detailed in **Appendix A** (Tables IX and X). This unified approach allows the DSE engine to autonomously mitigate architectural bottlenecks by activating

TABLE III: Full-Stack DSE Search Space Definition

Domain	Parameter	Values / Bounds
SW	`hd_dim`	$\{2048, 4096\}$
SW	`inner_dim`	$\{1024, 2048, 4096\}$
HW	`reram_size`	$\{128, 256\}$
HW	`out_channels_1/2`	$\{4, 8, 16\}/\{8, 16, 32\}$
HW	`frequency` (MHz)	$[80, 300]$
EDA	`syn_map_effort`	{low, med, high}
EDA	`enable_retime`	{false, true}
EDA	`enable_clock_gating`	{false, true}
EDA	`Exp_smartgen`	{none, timing, area}

aggressive synthesis strategies, thereby uncovering optimal Pareto points that isolated optimizations cannot reach.

IV. Experimental Setup and Results

This section validates the proposed multi-fidelity framework through three differentiated experiment groups. Due to time and license constraints (discussed in Section V), we employ fixed parameter configurations rather than BO-driven sampling to systematically probe the design space and demonstrate the framework's capability to extract high-fidelity PPA metrics across software, hardware, and EDA dimensions. A quantitative comparison with analytical-only baselines (e.g., Cimloop) or prior HD-PIM co-optimization (Yang et al.) is left for future work due to the lack of a common evaluation protocol. The evaluation environment and search space definitions are detailed in Section III. All experiments use PyTorch for Path 1, Synopsys Design Compiler and VCS for Path 2/3, and Cimloop for RRAM energy estimation, with *synth_mode=fast* and *top_module=hd_top*.

A. Metrics and Experimental Setup

Metrics are collected along a three-stage pipeline: Path 1 (Software Simulation) is mandatory for all design points; Path 2 (Logic Synthesis) executes after Gate 1 passes; Path 3 (Gate-Level Simulation) runs optionally after Gate 2. Data from later stages overwrites earlier estimates; PrimeTime PX (PtPX) overwrites Path 2 power when invoked. Table IV summarizes the primary metrics and their sources. Latency and energy are derived as

$$\text{Latency } (\mu\text{s}) = \frac{Cyc_{p3} \times 1000}{frequency} \tag{3}$$

$$\text{Energy}_{\text{p2}\times\text{p3}} \ (\mu\text{J}) = P_{p2} \times \text{Latency} \tag{4}$$

TABLE IV: Evaluation Metrics and Data Sources

Metric	Definition	Unit	Source
E_{p1}	SW energy (Path 1)	μJ	Cimloop
A_{p1}	SW area (Path 1)	mm^2	Cimloop
A_{p2}	Synth. logic area	μm^2	DC
P_{p2}	Post-synth. power	mW	DC
P_{p3}	Gate-level power	mW	PtPX
Cyc_{p3}	Gate-level cycles	—	VCS

All syntheses use the TSMC 40nm ULP PDK. The 15 design points are organized into three experimental groups: Group EDA (5 points) varies synthesis flags and effort levels, with P_{p2} as the primary metric; Group Arch (6 points) sweeps architectural scale and *inner_dim*, targeting E_{p1} and A_{p2}; Group Freq (4 points) sweeps target frequency from 225 to 300 MHz , with timing slack as the key constraint.

TABLE V: Group EDA: PPA vs. EDA Strategy

Exp	EDA	$\mathbf{A}_{p2}$	$\mathbf{P}_{p2}$
1	low	317,838	183.1
2	high/retime	318,773	183.3
3	med/power+gate_clock	316,191	180.0
4	med	317,838	183.1
5	**full**	**316,216**	**176.5**

TABLE VI: Group Arch: *inner_dim* vs. Cycles & Power

Exp	inner	oc1	oc2	$\mathbf{Cyc}_{p3}$	$\mathbf{P}_{p2}$	$\mathbf{E}_{\mathbf{p2}\times\mathbf{p3}}$
8	1024	8	16	2,398	183.1	2,195
9	2048	8	16	2,478	183.1	2,269
10	4096	8	16	2,650	183.1	2,426
11	1024	4	8	2,398	183.1	2,195

B. Group EDA

This group isolates the impact of fine-grained EDA flags by fixing the software and hardware architectural parameters.

With software and hardware parameters held constant, Path 1 metrics remain invariant across all experiments; the various EDA optimization flags do not affect the results of Path 1. Consequently, P_{p2} exhibits a range of 176.5–190.0 mW. Exp 5, which applies the full EDA optimization suite (high-effort mapping, retiming, clock gating, and power optimization), achieves the minimum dynamic power of 176.5 mW. This result empirically demonstrates that fine-grained EDA strategies yield measurable post-synthesis power reductions, an optimization dimension that remains inaccessible to pure-software simulation flows.

C. Group Arch

This group verifies whether Path 3 gate-level simulation accurately captures the hardware execution cycles influenced by pure software parameters (e.g., *inner_dim*).

As *inner_dim* scales from 1024 to 4096, the computational workload within the Encoder intensifies. The extracted Cyc_{p3} perfectly reflect this trend, monotonically increasing from 2,398 to 2,650 cycles. This confirms that Path 3 gate-level simulation, driven by the LFSR testbench, possesses the high fidelity required to accurately translate software parameter variations into foundational timing behaviors, providing highly precise energy estimations ($\text{Energy}_{\text{p2}\times\text{p3}}$).

D. Group Freq

To probe the hardware timing limits, the target clock frequency is aggressively scaled from 225 MHz to 300 MHz under a fixed architecture.

Path 1 software evaluation is agnostic to clock frequency; Path 2 logic synthesis imposes increasingly stringent timing constraints as the target frequency increases, resulting in a

TABLE VII: Group Freq: Frequency vs. Latency & Slack

Exp	Freq	Latency	P_{p2}	A_{p2}	*p2_slack*
12	225	10.65	206.0	317,836	0.61
13	250	9.59	228.9	317,711	0.26
14	275	8.73	251.8	317,718	0.02
15	300	7.99	274.9	320,705	0.00

monotonic degradation of *p2_slack* from 0.61 ns to 0.00 ns. Exp 15 demonstrates that the architecture meets timing closure at 300 MHz under the TSMC 40nm process, achieving a minimum inference latency of 7.99 μs.

E. Pareto Analysis and Design Insights

Analysis of the 15 evaluated design points reveals a Pareto-optimal subset comprising Exp 5, 9, 11, and 15. These candidates define the optimal boundaries of the Area-Latency-Power (ALP) design space, demonstrating that global optimality requires the simultaneous orchestration of algorithmic, architectural, and physical parameters.

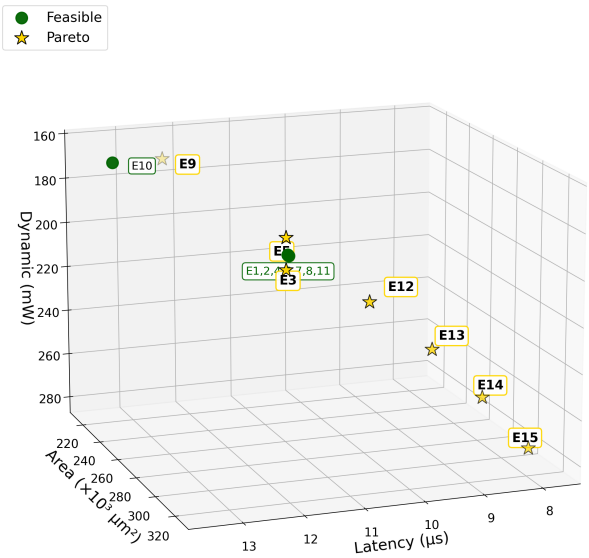


Fig. 3: Three-dimensional Pareto frontier (axes: A_{p2}, Latency, P_{p2}) illustrating the global design space.

The identified frontier highlights three distinct evaluation trajectories. Minimum power consumption is achieved by Exp 5 (176.5 mW), which utilizes aggressive synthesis directives and clock-gating strategies. In contrast, Exp 9 secures the minimum area footprint ($217,272$ μm^2) through architectural right-sizing of the encoder dimensions. Maximum performance is represented by Exp 15, which achieves a minimum inference latency of 7.99 μs by operating at the 300 MHz timing limit of the TSMC 40nm process.

The multi-dimensional trade-offs are visualized in Fig. 3 and Fig. 4. A critical observation is the non-linear scaling between performance and power; for instance, the transition from the balanced configuration (Exp 11) to the high-performance limit (Exp 15) yields a 33% reduction in latency at the cost of a 50% increase in dynamic power. Such insights validate the

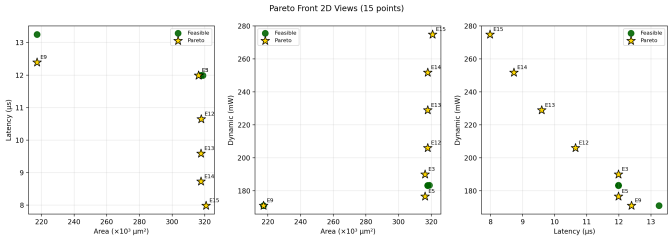


Fig. 4: Two-dimensional projected views of the multi-objective Pareto frontier, highlighting specific trade-off penalties.

utility of the proposed full-stack framework in quantifying steep physical penalties that are typically obscured in isolated software-level explorations.

F. Path 1 vs. Path 2 Area Fidelity Analysis

The integration of Path 2 (logic synthesis) is necessitated by a pronounced fidelity gap between software-level area estimations (A_{p1}) and physical gate-level realizations (A_{p2}). While Path 1 provides structural baselines, Path 2 exposes a vastly broader area variance driven by the complex interplay of architectural parameters and EDA synthesis directives, as illustrated in Fig. 5.

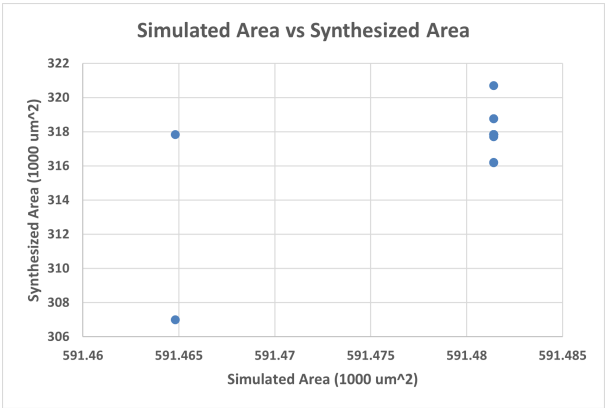


Fig. 5: Comparative analysis of Area Fidelity between Path 1 (Analytical) and Path 2 (Synthesis).

Crucially, our experiments reveal "Area-Timing Reversals" that invalidate pure software-level evaluations. Configurations deemed compact in Path 1 (e.g., smaller *hd_dim* or *inner_dim*) frequently exhibit a larger physical footprint (A_{p2}) post-synthesis. This ranking reversal, clearly evidenced in Fig. 5, occurs when stringent timing constraints compel EDA tools to insert high-drive-strength cells and extensive buffering to meet critical path requirements, completely offsetting any theoretical area advantages.

Furthermore, algorithmic hyperparameters intrinsically dictate baseline hardware resources. Parameters such as *reram_size* and *hd_dim* directly define the `ROW_ADDR_WIDTH` and the complexity of bus address decoders, triggering non-linear expansions in the total ASIC area. These coupled dependencies, alongside timing-driven area inflation, definitively prove that gate-level fidelity via

Path 2 is indispensable for identifying the true physical Pareto frontier.

V. Discussion and Conclusion

This section interprets the experimental results of Section IV, maps them to the predefined Research Questions, discusses implications and limitations, and concludes with future work directions.

A. Interpretation

Runtime Comparison: Experiments show that Path 1 (high-level simulation) takes 30 s, Path 2 (post-synthesis estimation) 360 s, and Path 3 (gate-level simulation) 400 s. These results highlight the trade-off between fidelity and runtime and indicate that neither software-only nor hardware-only evaluation can achieve optimal PPA; effective optimization requires a unified approach considering architecture, synthesis, and timing jointly.

Architecture and Inner Dimension: Group Arch experiments show that $Cyc{*}p3$ scales monotonically with *inner_dim* (2,398–2,650 cycles), and Exp. 9 achieves the minimum $A{*}p2$. These results confirm that the overall behavior of Path 2 and Path 3 matches design expectations and validates our simulation flows.

EDA Strategies: Under identical hardware constraints, P_{p2} exhibits a range of 176.5–190.0 mW. Exp. 5, which applies high-effort mapping, retiming, and clock gating, achieves the minimum power consumption. These optimizations are performed during synthesis and therefore cannot be accurately captured by high-level software simulation. The results empirically confirm that fine-grained EDA synthesis flags can effectively shift the Pareto frontier.

Frequency and Timing Limits: *p2_slack* monotonically decreases as the target frequency increases. Exp. 15 meets timing closure at 300 MHz under the TSMC 40nm ULP process without violations, validating the high-frequency limit of the proposed architecture. In contrast, the current software-level estimator (Timeloop) cannot accurately reflect timing behavior across different target frequencies, as it lacks gate-level timing analysis. Consequently, the practical frequency limit of the design can only be verified through synthesis and timing analysis.

The framework's core strength lies in modularity: the optimizer, evaluation pipelines, and threshold gates are decoupled and interchangeable. This enables adaptation to diverse design constraints and optimization targets. Black-box rapid synthesis coupled with Gate 1/2 early-stopping mechanisms prunes invalid evaluations, improving practical operability.

B. Limitations

1) Integration of Bayesian Optimization and Multi-Fidelity Pipeline: The study employs fixed-parameter configurations for systematic exploration; a fully integrated Bayesian Optimization (BO) scheme has yet to be implemented. Future work must address: (1) unified parameter mapping across software (*inner_dim*, *out_channels*), hardware (*reram_size*, frequency), and EDA flags, accommodating mixed discrete/continuous variables; (2) explicit path-triggering logic for Gate 1/2 thresholds and Pareto candidate filtering; (3) multi-fidelity or early-stopping mechanisms in BO to exploit the cost disparity between Path 1 and Path 2/3, maximizing Pareto coverage under constrained budgets.

2) Insufficient Accuracy Training and Pareto Validity: Due to time constraints, the software model was not trained to full convergence. The Gate 1 filtering mechanism, which relies on Path 1 accuracy, may exhibit distorted behavior; the reported accuracies (0.83–0.94) do not fully reflect the algorithm's capability. Under these premises, the current Pareto frontier serves primarily as a proof-of-concept for the multi-fidelity pipeline's feasibility rather than definitive design guidance. Future iterations must incorporate complete, multi-epoch training to ensure Pareto candidates yield practical design value.

3) Hardware Parameter Flexibility and Search Space Constraints: The baseline RTL architecture was not designed for arbitrary parametrization. Several critical parameters (encoder dimensions, memory interface widths, PE layouts) are statically bound, restricting the Design Space Exploration (DSE) scope. Future work requires refactoring the RTL with parameterized modules and configurable macros to support a broader search space. Moreover, Path 3 is architecturally designed to execute PrimeTime PX (PtPX) power analysis driven by actual waveform toggle behaviors, thereby capturing realistic circuit switching activities. Due to current licensing constraints, P_{p2} temporarily substitutes for gate-level power.

4) Power Analysis and PtPX Licensing Constraints: Lacking a PrimeTime PX (PtPX) license, the experiments substitute gate-level dynamic power with Path 2 synthesis reports (*report_power*). This static estimation does not capture realistic toggle behavior. Securing a PtPX license for waveform-driven dynamic power analysis in Path 3 is necessary for high-fidelity pre-silicon energy metrics.

5) Testbench Fidelity and Cycle Accuracy: The Path 3 testbench uses LFSR-generated stimuli rather than loading trained weights and test vectors from Path 1. Consequently, Cyc_{p3} does not represent precise end-to-end inference latency; it serves as a relative architectural reference for comparing execution variations across *inner_dim* configurations. Establishing seamless dataflow integration between Path 1 and Path 3 is required for accurate cycle and power estimation.

C. Conclusion and Future Work

This study proposes and implements a modifiable, automated full-stack hardware-software co-design framework for the HDnn-PIM architecture. We establish a three-stage evaluation pipeline (Path 1 $\rightarrow$ Path 2 $\rightarrow$ Path 3) and integrate fine-grained EDA synthesis flags into the search space. The core design philosophy is *modifiability*: optimizers, pipelines, thresholds, and data sources can be swapped as needed. Fifteen structured design points validate the framework's feasibility; black-box synthesis and Gate 1/2 early-stopping demonstrate practical operability. While limitations exist due to licensing and time constraints, this study validates the viability of the multi-fidelity evaluation pipeline and lays a foundation for comprehensive DSE and Pareto optimization.

TABLE VIII: Summary of Future Work Directions

Direction	Proposed Enhancements
BO & Multi-Fidelity	Unified SW/HW/EDA parameter mapping; path-triggering logic; multi-fidelity or early-stopping BO.
SW Training	Multi-epoch accuracy training for Gate 1 efficacy and Pareto frontier applicability.
RTL Parametrization	Parameterized modules and macros for encoder, memory, PE layout to broaden DSE scope.
PtPX Power Analysis	Replace Path 2 static power with PtPX waveform-driven analysis in Path 3 for high-fidelity energy metrics.
Dataflow Integration	Path 1–Path 3 testbench connection with trained weights and test vectors for accurate cycle and power estimation.

Table VIII summarizes the future work roadmap. Beyond these items, further research may extend the framework to physical design (floorplanning, placement, routing) and generalize the architectural scope to support broader accelerator classes (e.g., DNNs, GNNs).

REFERENCES

[1] A. Dutta, S. Gupta, B. Khaleghi, R. Chandrasekaran, W. Xu, and T. Rosing, "Hdnn-pim: Efficient in memory design of hyperdimensional computing with feature extraction," in *Proceedings of the Great Lakes Symposium on VLSI 2022*, 2022, pp. 281–286.

[2] A. Parashar, P. Raina, Y. S. Shao, Y.-H. Chen, V. A. Ying, A. Mukkara, R. Venkatesan, B. Khailany, S. W. Keckler, and J. Emer, "Timeloop: A systematic approach to dnn accelerator evaluation," in *2019 IEEE international symposium on performance analysis of systems and software (ISPASS)*. IEEE, 2019, pp. 304–315.

[3] T. Andrulis, J. S. Emer, and V. Sze, "Cimloop: A flexible, accurate, and fast compute-in-memory modeling tool," in *2024 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 2024, pp. 10–23.

[4] Q. Sun, T. Chen, S. Liu, J. Chen, H. Yu, and B. Yu, "Correlated multi-objective multi-fidelity optimization for hls directives design," *ACM Transactions on Design Automation of Electronic Systems (TODAES)*, vol. 27, no. 4, pp. 1–27, 2022.

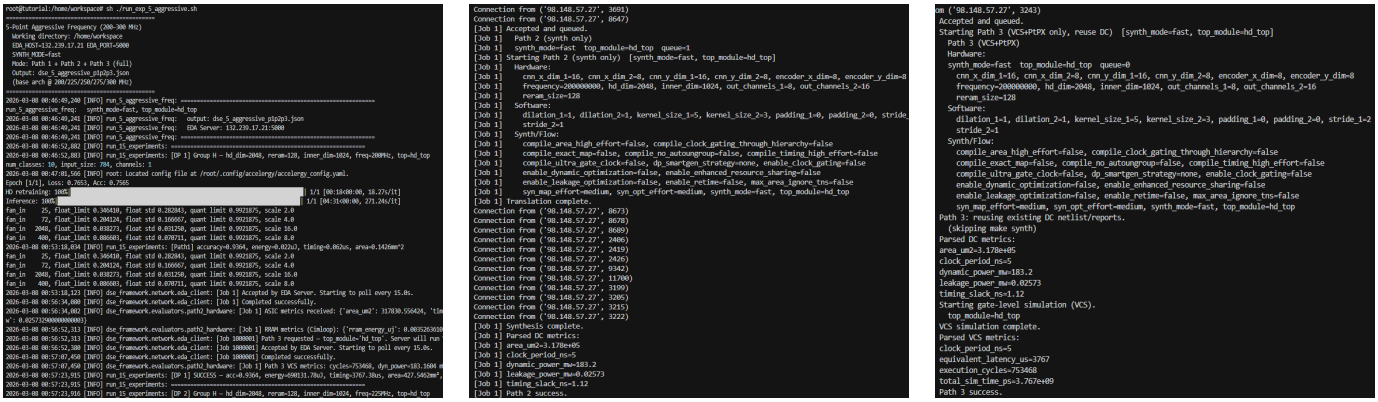
[5] S. Zheng, H. Geng, C. Bai, B. Yu, and M. D. Wong, "Boosting vlsi design flow parameter tuning with random embedding and multi-objective trust-region bayesian optimization," *ACM Transactions on Design Automation of Electronic Systems*, vol. 28, no. 5, pp. 1–23, 2023.

[6] C.-Y. Yang, M. Zhou, F. Ponzina, S. Sathya Prakash, R. Ayoub, P. Mercati, M. Subedar, and T. Rosing, "Multi-objective software-hardware co-optimization for hd-pim via noise-aware bayesian optimization," in *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*, 2024, pp. 1–9.

[7] S. Chen, C. Cai, S. Zheng, J. Li, G. Zhu, J. Li, Y. Yan, Y. Dai, W. Yin, and L. Wang, "Hiercgra: A novel framework for large-scale cgra with hierarchical modeling and automated design space exploration," *ACM Transactions on Reconfigurable Technology and Systems*, vol. 17, no. 2, pp. 1–31, 2024.

Appendix

A. Operational Traces and Log Parsing



((a)) Client-side orchestration. ((b)) Path 2: Synthesis trace. ((c)) Path 3: Simulation trace.

Fig. 6: System execution traces: (a) Optimization engine managing job queues and status polling; (b) Remote server executing logic synthesis and PPA parsing; (c) High-fidelity verification highlighting waveform-driven activity monitoring.

B. Hardware-Software Parameter Mapping

TABLE IX: Cross-Layer Parameter Translation and Hardware Implementation Constraints

Target Primitive	**DSE Parameter**	**Hardware Mapping Logic and Constraints**
`RRAM_ROW_ADDR_WIDTH`	`reram_size`	$\lceil \log_2(\text{reram_size}) \rceil$; must be 2^n for decoder.
`HV_LENGTH`	`hd_dim`	Direct mapping; upper bound 8191 (HAMMING_DIST_WIDTH=13).
`INNER_DIM`	`inner_dim`	$N_{\text{RF}} = \text{inner_dim}/32$; $W = \lceil \log_2(N_{\text{RF}}) \rceil$.
`CNN1_INPUTS_NUM`	`cnn_x_dim_1` × `cnn_y_dim_1`	Scalar product.
`CNN2_INPUTS_NUM`	`cnn_x_dim_2` × `cnn_y_dim_2`	Scalar product.
`ENC_INPUTS_NUM`	`encoder_x_dim` × `encoder_y_dim`	Product; encoder receptive field.
`HV_SEG_WIDTH`	`hd_dim` / `ENC_INPUTS_NUM`	Must divide; HV_SEG_WIDTH $\geq$ 20; 256 mod HV_SEG_WIDTH = 0.
`TB_CLK_PERIOD_NS`	`frequency`	Same; for LFSR testbench clock generator.
`ELABORATE_ROOT`	`top_module`	core or hd_top; selects testbench.

C. Synthesis Optimization Space

TABLE X: Definitions of Fine-Grained Synthesis Flags and Mapping to EDA Directives

DSE Control Knob	**Data Type**	**Target Design Compiler (DC) Implementation Directive**
`max_area_ignore_tns`	Boolean	`set_max_area 0 -ignore_tns` (area over TNS).
`enable_retime`	Boolean	`compile_ultra -retime`
`compile_timing_high_effort`	Boolean	`compile_ultra -timing_high_effort_script`
`compile_area_high_effort`	Boolean	`compile_ultra -area_high_effort_script`
`compile_ultra_gate_clock`	Boolean	`compile_ultra -gate_clock`
`compile_exact_map`	Boolean	`compile_ultra -exact_map`
`compile_no_autoungroup`	Boolean	`compile_ultra -no_autoungroup`
`enable_leakage_optimization`	Boolean	`set_leakage_optimization true`
`enable_dynamic_optimization`	Boolean	`set_dynamic_optimization true`
`enable_enhanced_resource_sharing`	Boolean	`set compile_enhanced_resource_sharing true`
`Exp_smartgen_strategy`	Categorical	`set_Exp_smartgen_options -optimization_strategy [timing|area]`